

Data Set

I frequently work with public health-related data as part of my job, and was interested in the application of machine learning in a healthcare context. The data set I identified¹ consists of deidentified information extracted from electronic health records for patients with diabetes who were hospitalized between the years of 1999 and 2008. The data were originally published alongside a journal article² which used logistic regression to analyze the impact of blood glucose measurement (`AlCresult` in the dataset) during hospitalization on readmission within 30 days, controlling for admission-related factors (category of primary diagnosis and specialty of healthcare provider referring the patient), discharge disposition (i.e. where the patient was sent after being discharged from the hospital), and demographic variables (age and race of the patient – gender was also considered but dropped from the analytic model as insignificant).

However, the shared data set contained a number of variables which were not analyzed in the original paper, and it seemed plausible that machine learning techniques could identify more complex relationships between patients' medical history and early hospital readmission, especially given the large amount of data available (101,704 records). Table 1 below summarizes the data available.

Variable(s)	Type	Range / # Values	Description
readmitted	Categorical	3	"<30" if readmitted within 30 days, ">30" if readmitted after 30 days, "NO" if not readmitted
encounter_id	Integer	12522 - 443867222	Unique identifier for each observation
Patient Data			
patient_nbr	Integer	135 - 189502619	Unique identifier for each patient. There may be multiple observations in the data set corresponding to the same patient, if they had multiple hospitalizations meeting the inclusion criteria for the dataset.
race	Categorical	5	Options are Caucasian, African American, Hispanic, Asian, other. May also be missing (unknown race).
age	Categorical	10	Provided in 10-year bins e.g. [20, 30)
gender	Categorical	3	Male, female, unknown/other
weight	Categorical	10	Provided in 25-kg bins e.g. [50, 75). Frequently missing.
payer_code	Categorical	23	Insurance type. Frequently missing.
Hospital Encounter Data			
admission_type_id	Categorical	9	Category of hospital admission e.g. emergency, elective

¹ Clore, J., Cios, K., DeShazo, J., & Strack, B. (2014). *Diabetes 130-US Hospitals for Years 1999-2008* [Dataset]. UCI Machine Learning Repository. <https://doi.org/10.24432/C5230J>.

² Strack, B., DeShazo, J. P., Gennings, C., Olmo, J. L., Ventura, S., Cios, K. J., & Clore, J. N. (2014). Impact of HbA1c measurement on hospital readmission rates: Analysis of 70,000 clinical database patient records. *BioMed Research International*, 2014(1), 781670. <https://doi.org/10.1155/2014/781670>.

admission_source_id	Categorical	21	How the patient came to be hospitalized, e.g. transfer from another hospital, referral by primary care provider, emergency room
discharge_disposition_id	Categorical	29	Where the patient was sent after being discharged, e.g. home, long-term care
time_in_hospital	Integer	1 - 14	Days spent in hospital. The original research which resulted in this dataset excluded patients who were hospitalized more than two weeks or less than one day.
medical_specialty	Categorical	84	Specialty of doctor admitting the patient to hospital, e.g. internal medicine, cardiology. Frequently missing.
diag_1, diag_2, diag_3	String	~1000	ICD-9 diagnostic codes corresponding to up to primary diagnosis for which the patient was admitted and up to two secondary diagnoses. While all patients in the sample were diabetic, this was not necessarily the primary reason they were hospitalized.
number_diagnoses	Integer	1 - 16	Total number of diagnoses in the patient's chart (not necessarily all related to current hospitalization).
Tests and Medications			
num_lab_procedures	Integer	1 - 129	Number of lab tests run during hospitalization
num_procedures	Integer	0 - 6	Number of other procedures (e.g. scans, surgery) performed
max_glu_serum	Categorical	4	"None" if serum blood glucose was not measured; "normal", ">200", or ">300" depending on test result if it was measured.
A1Cresult	Categorical	4	"None" if A1C (long-term blood glucose) was not measured; "normal", ">6", or ">7" depending on test result if it was measured
num_medications	Integer	1 - 81	Number of distinct medications administered
diabetesMed	Boolean	-	Whether the patient was taking any medication for diabetes
change	Boolean	-	Whether any new medications were prescribed OR dosage of current medications was adjusted
[24 specific diabetes medication variables]	Categorical	4	"None" if medication not prescribed, "Steady" if prescribed and dosage unchanged, "Up" or "Down" if dosage changed
Recent Medical History			
number_outpatient	Integer	0 - 40	Number of times the patient was seen in an outpatient setting in the year leading up to the hospitalization
number_inpatient	Integer	0 - 19	As above but for inpatient setting
number_emergency	Integer	0 - 54	As above but for emergency setting

Data Cleaning and Processing

While admission source, admission type, and discharge disposition were meant to be categorical variables, they were stored as numeric ID codes which needed to be matched to a separate spreadsheet (included with the data set) to obtain the description for each category. Several of the possible values for discharge disposition indicated that the patient was "expired" (deceased) or transferred to hospice. Since it would be pointless to predict whether patients who have passed away will be readmitted to the hospital, these observations were dropped. Additionally (perhaps because the data set was constructed by combining information from

multiple Electronic Health Record systems), several categories for each of these variables were redundant with one another and needed to be combined. For example, three of the category labels for admission type were “NaN”, “Not Available,” and “Not Mapped,” which were combined into an “Unknown” category.

The data set contained multiple encounter observations for some patients, reflecting the fact that these patients were hospitalized multiple times over the course of the 10 years spanned by the data set. While it is reasonable to expect that a patient’s past encounters would be predictive of their outcomes, retaining all observations for each patient would violate the independence assumptions underlying many machine learning algorithms. To preserve independence while incorporating patients’ hospitalization history into the data being analyzed, I created several features based on previous encounters, then retained only the final episode for each patient. While the data set did not include dates, it could be sorted by encounter ID to chronologically order encounters.

After sorting the dataset, I constructed several features for each encounter based on the patient’s previous encounters, if any: number of previous encounters, total time previously spent in hospital (sum of previous `time_in_hospital` values), previous readmissions within 30 days, and distinct ICD-9 codes ever appearing as primary or secondary reasons for admission (as a measure of the number of serious conditions affecting the patient, which could differ from the total number of diagnoses in their chart as measured by the existing `number_diagnoses` variable). These features needed to be constructed *before* splitting the data into train and test sets, since basing e.g. a number of past admissions on only a fraction of the dataset would underestimate the true value. All other data processing could be done after splitting the train and test sets.

The ICD-9 codes for patients’ primary diagnosis and the medical specialty of the admitting physician seemed relevant to patient outcomes, but had too many potential values (many of which occurred only a small number of times) to be useful if directly one-hot encoded. For medical specialty, I combined closely-related or redundant category labels (e.g. “Surgeon” and “Surgery”) and lumped uncommon categories which could not be combined into an “Other” label. For diagnoses, I used a publicly available JSON file mapping ICD-9 codes to broader disease categories³ to categorize the primary diagnosis into one of 13 groups. Additionally, I created a binary variable indicating whether the primary diagnosis was diabetes, given that several of the other variables included in the data set (such as blood glucose and specific medications) would likely be more relevant if diabetes was the primary reason for hospitalization, as opposed to a comorbid condition during treatment for a different issue.

The last major issue addressed during data cleaning was missing values. Nearly all observations were missing values for the `weight` variable, and it seemed unlikely that the variables with full information would correlate strongly enough with weight for reliable imputation, so it was dropped.

³ <https://github.com/sirrice/icd9>

The blood glucose measurements, `max_glu_serum` and `A1Cresult`, were also missing for a large majority of observations. However, the original paper identified whether or not A1C was measured -- not just the result -- as clinically relevant. To include this information in the model, I chose to replace missing values with "Not measured" for these two variables. Both were already categorical (`max_glu_serum` takes values None, 'Norm', '>200', and '>300', and `A1Cresult` takes values None, 'Norm', '>6', and '>7'), so adding an additional category for patients with no measurement before one-hot encoding seemed reasonable.

`race`, `medical_specialty`, and `payer_code` (insurance type) also had high rates of missingness. I initially chose to handle these similarly by creating an "Unknown" category to complement the existing categories, since I believed that where the information was available it could be valuable in predicting patient outcomes.

Finally, I created a binary version of the target variable (hospital readmission) to specifically identify readmission within 30 days (combining later readmission or no readmission into one class), since the authors of the original study indicated that 30 days was a relevant threshold for clinical and insurance contexts.

Data Exploration

The target class distribution was quite unbalanced -- only around 5% of observations had the outcome that would be most valuable to predict (readmission within 30 days). Around 15% were readmitted after more than 30 days, while the remaining ~80% were not readmitted.

Examining the distribution of values for each variable, it became clear that many of the variables referring to specific medications were likely to be uninformative or lead to overfitting, as very few patients were taking those medications. For example, only two patients in the training set, out of over 55,000, received troglitazone during their admission, and none received examide or citoglipton. I attempted to mitigate this issue by only retaining medication variables as features if more than 1000 patients in the training set received the medication (that is, had values other than 'No').

Visualizing the numeric variables revealed that many of them were skewed or included outliers. For instance, almost all patients had fewer than 5 emergency encounters in the year preceding their hospitalization, but a few had a considerably higher number. Applying log scaling to these variables made the outliers less extreme, but the distribution remained skewed. I first tried using scikit-learn's `PowerTransformer` on top of log-scaling to scale the numeric features. This scaler minimizes skewness in distributions using a function called the Yeo-Johnson transform and is more robust to outliers than scalers that rely on the range or standard deviation of the data.⁴ Unfortunately, even this did not completely resolve skewness issues, which likely contributed to less-than-ideal PCA and clustering results in Project 3. I eventually reverted to

⁴ https://scikit-learn.org/stable/auto_examples/preprocessing/plot_all_scaling.html

using the `StandardScaler`, which scales the data to a mean of 0 and variance of 1, on the log-scaled numeric data.

Graphing the distribution of target variable values against each of the features, I found that most of the variables were only very weakly associated with the target class (the distribution of target class values looked very similar across values of the feature). In particular, most of the medication variables as well as race, gender, and the diabetes-specific measures (serum glucose and A1C) appeared to have nearly no predictive value. I retained these features for initial model fitting, since it seemed possible that they could have interactions with other features even if they were not predictive alone.

Supervised Learning

Given how unbalanced the target values are, I opted to use models that could take the `class_weight='balanced'` parameter, which weights observations based on the rarity of the target value. This improves predictions for less common target classes, and in this particular case the class we are most interested in predicting -- readmission within 30 days -- is very uncommon. Without accounting for the imbalance, many models will simply assign most/all observations to the more common class, resulting in a very high accuracy but very low recall in the case we most care about. Given that failing to predict an adverse outcome is a much more serious issue than overall accuracy, I was specifically interested in optimizing for recall on the minority class, even if this resulted in a lower accuracy or F1 score for the model overall.

Models which work with `class_weight='balanced'` include logistic regression, decision trees, random forests, and support vector machines (SVM). I chose to use logistic regression as a simple baseline model, because it was used (with fewer features) in the original paper, although I anticipated that the non-normality of some numeric features could limit its effectiveness. Logistic regression predicts the probability that an observation belongs to a particular class as a function of the feature values, each of which is assigned a coefficient, and these coefficients can be regularized (shifted towards zero in the case of ridge regression, or zeroed out entirely in the case of lasso regression) to improve generalizability.

I selected Random Forest as a more complex model which nonetheless scales reasonably to large datasets and which is less impacted by non-normality of features. Random forests fit a large number of decision trees, each fit to a slightly different subset of the observations and/or features, with the predictions of all trees averaged to produce the forest's prediction. (I experimented slightly with SVM as well, but it proved to be impractically slow for my large dataset).

For each model, I began by testing both multi-class and binary classification, to see whether attempting to additionally predict readmission in more than 30 days detracted from the quality of predictions for the most important outcome of readmission within 30 days. For multiclass classification the highest recall for this class with default hyperparameters was 0.44, whereas for

binary classification it was 0.64, so for hyperparameter tuning I chose to focus on binary classification in the hopes of achieving better performance.

The logistic regression model obtained a recall of 0.64 on the binary classification task with default hyperparameters. The only hyperparameters which can be tuned for this model are the regularization parameter `C` and the type of penalty used (L1 a.k.a. Lasso, which shrinks the coefficients of features least related to the target to zero, L2 a.k.a. Ridge, which limits the magnitude of coefficients but does not eliminate features, or a mix of both using the `l1_ratio` parameter with the `elasticnet` penalty option). For grid search, I tested a wide range of `C` values (powers of 10 ranging from 0.01 (very strong regularization) to 100 (very weak regularization)) and `l1_ratio` values of 0 (L2 penalty only), 0.25, 0.5, 0.75, and 1 (L1 penalty only). However, the performance of the model was essentially unchanged regardless of the hyperparameter values.

With default parameters, Random Forest obtained perfect classification on the training set and 95% accuracy on the test set, but did so by classifying virtually all observations as not readmitted (a recall of 0% on the relevant class). That is, the model overfit the training data, leading to poor generalization. Random Forests have a large number of hyperparameters available for tuning, including the number of estimators (trees) fit, the splitting criterion (Gini coefficient, entropy, or log-loss), maximum depth, maximum leaf nodes, minimum samples or fraction of weight per leaf, and number of features to consider at each split. Other than the splitting criterion and number of estimators, all of these features have the effect of limiting the size of the decision trees created. When experimenting with values, I found very little difference in performance with different splitting criteria, and that tweaking multiple hyperparameters associated with tree size did not seem to be much more effective than choosing one to adjust in this particular case. In the final submission, I varied the values of `max_depth` and `min_weight_fraction_leaf`. Additional parameters tested, but for which results are not included in the submitted notebook, included `criterion`, `num_features`, and `num_estimators`.

The best-performing combination (maximum depth 15 with a minimum leaf weight fraction of 0.01) achieved a recall of 0.67 in cross-validation and 0.69 on the test set. While this is far better than the random forest performance with default parameters, it is only slightly better than the performance of the less sophisticated logistic regression model. Moreover, the false-positive rate was very high (recall of only 64% for non-readmission within 30 days, or a false-positive rate of 36%). This is likely not sufficient performance to justify using this classification algorithm in a clinical setting.

I additionally experimented with undersampling the majority class or upsampling the minority class to mitigate class imbalance, but this did not meaningfully improve the recall achievable for readmission within 30 days. This is likely due to the low correlations between feature values and the target class – if no features or interactions between features are strongly associated with a particular outcome, it is difficult to reliably predict outcomes. Later, I also experimented with dropping the least-correlated features in the hopes that reducing noise in the dataset would lead

to a better-fitting model, but this also did not improve model performance (although it did not have a strong negative impact either).

PCA for Feature Selection

Principal Components Analysis (PCA) transforms data onto a new set of dimensions (“principal components”) that capture as much of the variance in the dataset as possible. Highly correlated variables can often be described well by a lower number of components.

Running PCA on the full training set required a very large number of variables (42) to capture 95% of the variance in the data, most of which only loaded heavily on the features corresponding to one of the one-hot encoded categorical variables. That is, these variables were not strongly associated with any of the other features. Because they were also negligibly associated with the target value, I chose to drop them at this stage, which reduced the number of columns in the training set from 109 to 39, and the number of principal components needed to explain 95% of the variance from 42 to 18. The recall for the class of interest dropped slightly, to 0.66, after this change. The two components which explained the largest proportion of variance were associated most strongly with medical history (e.g. previous readmissions, previous time spent in hospital, inpatient visits in the past year) and medical treatment received during the current admission (e.g. time in hospital, number of medications, number of procedures).

Initially, fitting the highest-performing model from supervised learning on the PCA-transformed data yielded significantly worse recall (0.56) for readmission within 30 days. I suspected that the model hyperparameters need to be adjusted to compensate for the lowered number of features, since the complexity needed to handle a large number of features could well be too much for (and thus overfit) the simplified feature set. After repeated trial-and-error, I found that raising `min_weight_fraction_leaf` from 0.01 to 0.035 raised recall on the target class to 0.65, nearly matching the performance of the original model. While I was not able to achieve higher performance, an equally performant model which runs more quickly due to the lower number of features is still valuable.

Clustering

Initial attempts to fit clustering algorithms on the data exceeded the Google Colab memory limits, because the data set was large (~55,000 observations) and both hierarchical and DBSCAN clustering require computation of distances between all pairs of observations. Randomly sampling half of the training set and using approximation techniques (e.g. ball trees rather than fully-accurate distance calculations) reduced the computational demands enough to allow clustering models to be fit, but performance was extremely poor.

The inertia plot for the k-means algorithm did not show a clear “elbow” either with the original data set or the PCA-transformed data, and k-means models fit with $k = 4, 3$, and 2 all failed to produce clusters associated with the target class, as indicated by ARI values near 0. Moreover, the separation between clusters was very poor (indicated by low silhouette scores). K-means,

which assigns points to the cluster with the nearest centroid, is not the only algorithm for clustering, but agglomerative clustering (which initializes each point as its own cluster and merges the clusters closest to each other until the desired number of clusters is reached) and DBSCAN (which builds clusters around “core points” in high-density regions of the parameter space) performed just as poorly.

I believe several factors contributed to the inability to effectively cluster the data. The size of the data set necessitated approximation rather than full distance calculations, but perhaps more importantly the majority of the features did not follow a normal distribution of values, even after scaling. Some were one-hot encoded categorical variables, which can only take two values, and even those based on numeric variables could not be transformed to a normal distribution and were limited in the values they could take. This constrains the possible distances between observations, and thus the separation between clusters that can be formed. The extremely poor ARI values likely occur for the same reason as poor predictive performance in supervised learning, namely weak correlation between features and the target value – there are simply not particular combinations of feature values that are reliably associated with readmission within 30 days.

Discussion

I believe my biggest mistake was in dataset selection. I erroneously assumed that a dataset with a large number of observations and features would be well-suited to machine learning-based classification, even if the correlations between each individual feature and the target value were weak, because I expected that there would be more complex relationships between features that were more predictive of the target. If such relationships had been present, a random forest model would have been well-suited to detecting them. Constructing additional numeric features based on the information available in the dataset (such as number of previous visits and distinct diagnoses) was worthwhile (since there were some of the more predictive variables), but these features were also not sufficient to achieve high recall on the relevant outcome, and their skewness may have contributed to poor clustering performance. However, it is worth noting that high skewness which could not be resolved by scaling was also an issue with some of the few numeric variables included in the original data.

I also regret waiting until the final stage of the project to drop uninformative features and reduce the number of observations in the dataset. While this would not have improved performance, it would have significantly reduced the amount of time required to fit each model, which would have allowed more types of models, and more combinations of parameters, to be tested. For example, SVM and gradient-boosted random forests were time-prohibitive to fit on the full dataset. (Given the issues of feature skew and weak correlations to the target value, I suspect these models would also have performed poorly, but it would have been worthwhile to at least test them.)